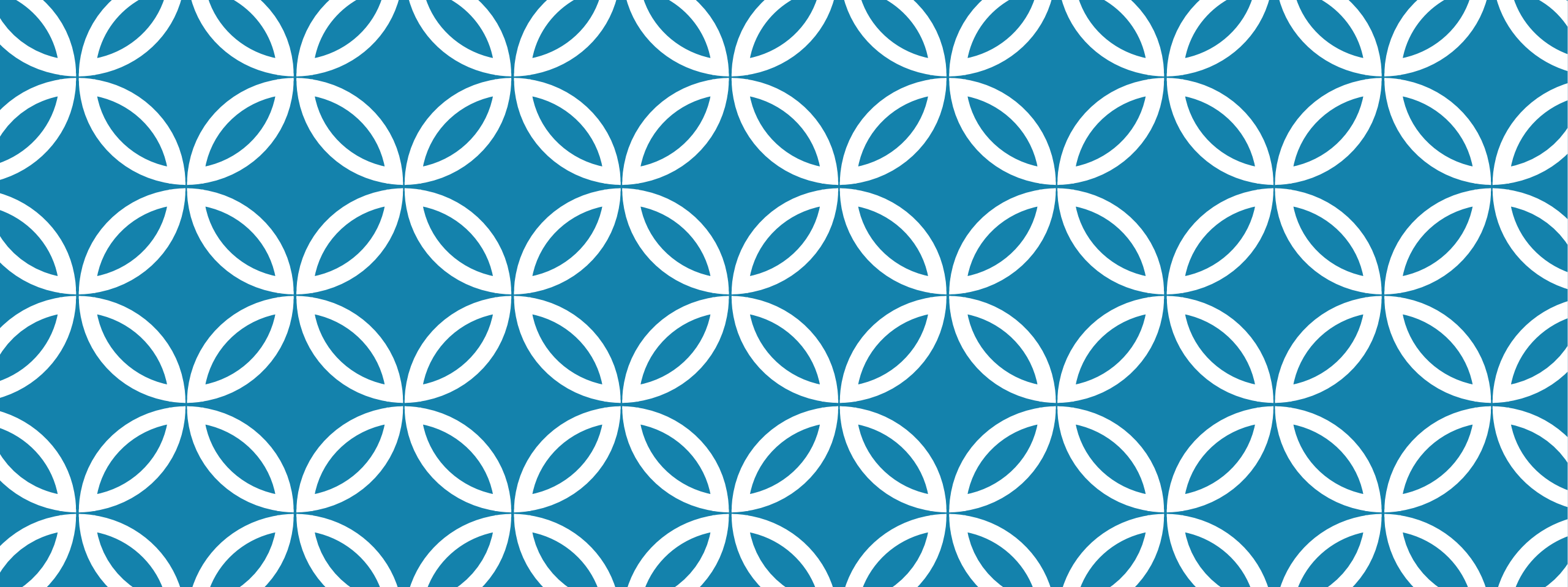




OPTICAL DATA TRANSMISSION

MEMBERS

Aadarsh

Mohanan-1032231417

Abeer Choudaha

-1032233636

INDEX

- **Introduction**
- **Literature Review**
- **Block Diagram with working**
- **Interfacing Diagram**
- **Hardware Components**
- **Software Implementation**
- **Results**
- **Conclusion**
- **References**

INTRODUCTION

What is Optical Data Transmission?

- It is a form of wireless communication, also known as Li-Fi (Light Fidelity) or Optical Wireless Communication (OWC).
- It transmits data by modulating the intensity of light (i.e., flashing LEDs very quickly).
- The data is received by a photodetector, which converts the light pulses back into an electrical signal.

Project Goal

- To design and build a functional, one-way data link to wirelessly transmit a text string ("@Optical Data Transmission@") from a transmitter (ESP8266) to a receiver (ESP32).

INTRODUCTION

Core Method

- Transmitter: An ESP8266 controls Infrared (IR) LEDs.
- Receiver: An ESP32 uses a solar panel as a sensor.
- Modulation: We use On-Off Keying (OOK) – the simplest form of digital modulation, where '1' is Light ON and '0' is Light OFF.
- Decoding: The receiver applies Digital Signal Processing (DSP) concepts to reliably sample, filter, synchronize, and decode the light signal.

LITERATURE REVIEW

Two primary modulation methods for this project:

1. Frequency Shift Keying (FSK) - (Initial Approach)

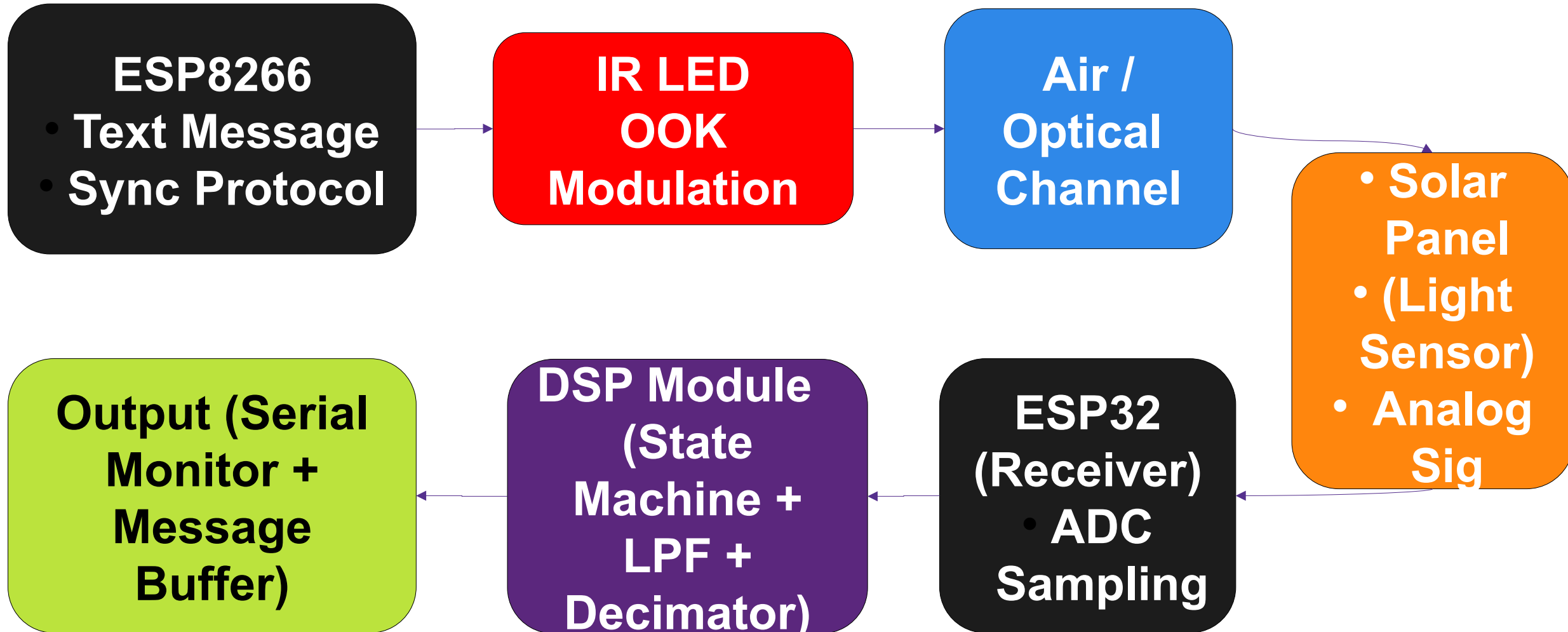
- Concept: Encode '0' and '1' as different flashing frequencies (e.g., 15 Hz vs. 65 Hz).
- DSP Method: Use two sharp Band-Pass FIR Filters on the receiver to distinguish which frequency is present.
- Result: FAILURE. Our analysis (using FFT and FIR debug code) showed this method was unreliable with our hardware.
- Reason: Harmonic Interference (Cross-talk). The transmitter's square wave (flashing) signal contains harmonics (e.g., the 15 Hz signal also has energy at 45 Hz, 75 Hz). These harmonics falsely triggered the wrong filters, making it impossible to distinguish '0' from '1'.

LITERATURE REVIEW

2. On-Off Keying (OOK) - (Final, Successful Approach)

- Concept: Encode '0' as Light OFF and '1' as Light ON.
- Initial Problem: Our first tests (with red LEDs) failed due to a low Signal-to-Noise Ratio (SNR). The "ON" signal was too weak to be seen over the background noise.
- Solution:
 - Hardware: Switched to Infrared (IR) LEDs, which provide a much stronger signal to the solar panel (as its peak sensitivity is in the IR spectrum).
 - Software: Implemented a Synchronization Protocol (a "Start Pulse") to fix timing errors between the transmitter and receiver.
- DSP Method: Use a Low-Pass FIR Filter (Moving Average) to reject high-frequency noise and clearly measure the "ON" or "OFF" energy level for each bit.
- Result: SUCCESS. This simpler, more robust method worked reliably.

Block Diagram with Working



Block Diagram with Working

- **Transmitter (ESP8266)**
- **Packetize:** Takes the string "Optical Data Transmission" and wraps it with @ markers.
- **Loop:** For each character in the packet:
- **Sync:** Sends a 1.5-second **Start Pulse** (Light ON) followed by a 0.5-second **Gap** (Light OFF).
- **Modulate (OOK):** Sends the 8 bits of the character.
 - For '1': Light **ON** for 500ms.
 - For '0': Light **OFF** for 500ms.
- **Pause:** After the full packet is sent, it waits for 5 seconds before repeating.

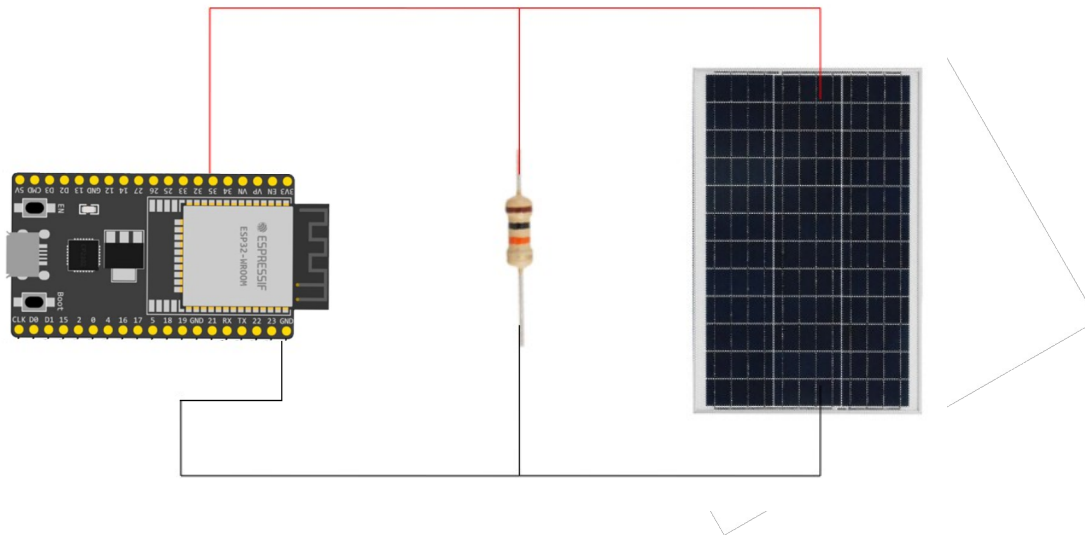
Block Diagram with Working

Receiver (ESP32)

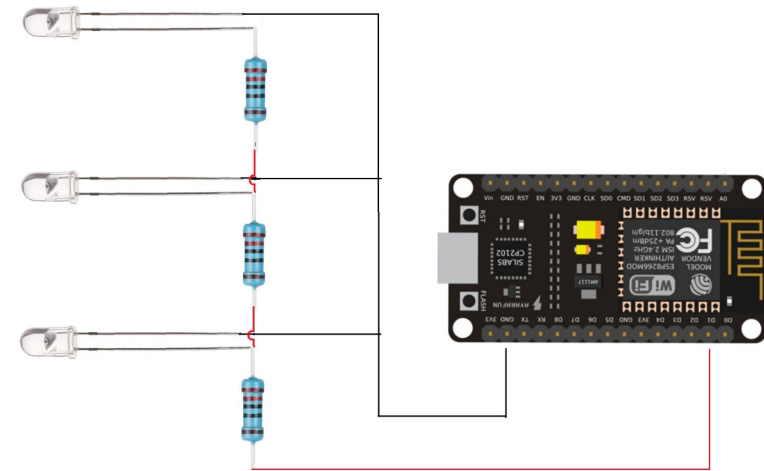
- **Sample:** The ESP32's ADC samples the solar panel's voltage at **2000 Hz**.
- **Wait for Sync:** A state machine averages samples in 500ms windows (decision_point), waiting for a signal stronger than START_PULSE_THRESHOLD.
- **Synchronize:** When the Start Pulse is detected, the receiver skips the next 3 windows (the rest of the pulse + the gap) to perfectly align with the first data bit.
- **Filter & Decode:** For the next 8 windows (bits):
 - It applies a **Moving Average (Low-Pass FIR Filter)** to the 1000 samples to get a stable average_light_level.
 - It compares this average to the ON_OFF_THRESHOLD.
 - If Avg > Threshold -> Decides '1'.
 - If Avg < Threshold -> Decides '0'.
- **Store & Print:** It assembles the 8 bits into a character. If it's between the @ markers, it's stored in a buffer. When the end @ is received, the **final, complete message is printed**.

Interfacing Diagram

RECEIVER



TRANSMITTER



Interfacing Diagram

Transmitter Circuit (ESP8266)

- **Components:** ESP8266, 3x IR LEDs, 3x 220 Ω Resistors
- **Connections:**
 - The 3 IR LEDs are connected in **parallel**.
 - Each LED's anode (+) is connected through its own 220 Ω resistor.
 - All three resistor branches are connected to **Pin D1 (GPIO 5)** of the ESP8266.
 - All three LED cathodes (-) are connected to **GND**.

Receiver Circuit (ESP32)

- **Components:** ESP32, Solar Panel, 1x 10k Ω Resistor
- **Connections:**
 - The **Solar Panel's RED (+) wire** connects to **Pin 35** (an ADC pin).
 - The **Solar Panel's BLACK (-) wire** connects to **GND**.
 - The **10k Ω resistor** is connected in parallel to the panel (from **Pin 35** to **GND**) to act as a pull-down and prevent a floating pin.

Hardware Components

Component	Function	Role in System	Characteristics
ESP8266 (NodeMCU)	Transmitter Controller	Generates the timed pulses for '0's, '1's, and sync.	3.3V logic, programmable in C++ (Arduino IDE).
ESP32	Receiver Controller & DSP	Samples the sensor, filters the signal, and decodes the bits.	Dual-core, 12-bit ADC (<code>analogRead</code>), 3.3V logic.
Infrared (IR) LEDs	Light Emitter	Converts electrical '1's into optical (light) ON signals.	~940nm wavelength. Matches the peak sensitivity of the silicon solar panel, providing a very strong signal.
Solar Panel (5V)	Light Detector (Sensor)	Converts incoming IR light pulses into a measurable analog voltage.	Acts as a large-area photodetector. Its slow response time was a problem for FSK, but acceptable for OOK.
10kΩ Resistor	Pull-Down Resistor	Provides a path to ground for the ADC pin.	Ensures the ADC reads a stable 0 when the LEDs are OFF, fixing the "floating pin" issue.
220Ω Resistor (x3)	Current-Limiting Resistor	Protects the LEDs and the ESP8266 GPIO pin from damage.	Limits current to a safe level for each of the three parallel LEDs.

Software Implementation Code

```
// --- TRANSMITTER V_LOOK_PACKET (Sends String Packet) ---
```

```
const int LED_PIN = 5; // D1
```

```
String message = "Optical Data Transmission";char startMarker = '@';char endMarker = '@';
```

```
int bit_duration = 500; // 500ms per bit
```

```
void setup() { pinMode(LED_PIN, OUTPUT); digitalWrite(LED_PIN, LOW); // Start with LED OFF  
Serial.begin(115200); // Optional, for monitoring }
```

```
void loop() { Serial.println("--- Sending New Packet ---"); // 1. Send the Start Marker '@'  
Serial.print("Sending Start Marker: @"); send_char(startMarker); Serial.println(" ...Done.");
```

```
    // 2. Send the message, character by character for (int i = 0; i < message.length(); i++) {  
    char c = message.charAt(i); Serial.print("Sending Char: "); Serial.print(c); send_char(c);  
    Serial.println(" ...Done."); } }
```

```
    // 3. Send the End Marker '@' Serial.print("Sending End Marker: @"); send_char(endMarker);  
    Serial.println(" ...Done.");
```

```
    // 4. Wait for 5 seconds (silence) Serial.println("--- End of Packet. Pausing... ---\n");  
    digitalWrite(LED_PIN, LOW); delay(5000); }
```

```
// This function sends one full character (8 bits)void send_char(char c) { // 1. Send a Start  
Signal (for this character) digitalWrite(LED_PIN, HIGH); delay(1500); // 1.5s ON pulse // 2.  
Send a Separation Gap digitalWrite(LED_PIN, LOW); delay(500); // 0.5s OFF gap
```

```
    // 3. Send the 8 bits of the character for (int i = 7; i >= 0; i--) { // MSB first if (bitRead(c, i)  
    == 1) { digitalWrite(LED_PIN, HIGH); // LEDs ON for '1' } else { digitalWrite(LED_PIN,  
    LOW); // LEDs OFF for '0' } delay(bit_duration); // Hold for 500ms } }
```

```
// --- RECEIVER V_LOOK_PACKET (Receives String Packet) ---
```

```
const int SENSOR_PIN = 35; // Make sure this is 35!
```

```
// --- TUNE THESE VALUES ---const int ON_OFF_THRESHOLD = 200; // Set halfway between OFF (0) and stable ONconst int  
START_PULSE_THRESHOLD = 200; // Set to a reliable 'ON' level
```

```
// --- Packet Markers ---const char START_MARKER = '@';const char END_MARKER = '@';String messageBuffer = ""; // Holds the incoming  
messagebool receivingMessage = false; // Flag
```

```
// --- Sampling parameters ---const unsigned long timeBetweenSamples = 500; // 500 µsconst int decision_point = 1000; // 500ms window
```

```
// --- State ---unsigned long lastSampleTime = 0;double accumulator = 0.0;int sample_count = 0;byte receivedByte = 0; // We build the 8-bit character  
here
```

```
// --- State Machine Definition ---enum State { WAITING_FOR_START, SKIPPING_GAP, READING_BITS };State currentState =  
WAITING_FOR_START;int bit_counter = 0;const int MESSAGE_LENGTH = 8;int windows_to_skip = 0;
```

```
void setup() { Serial.begin(115200); Serial.println("OOK String Receiver Ready. Waiting for Start Marker '@'..."); lastSampleTime = micros(); }
```

```
void loop() { // --- Sampling Timer --- if (micros() - lastSampleTime < timeBetweenSamples) { return; } lastSampleTime += timeBetweenSamples;
```

```
    // --- Accumulate Signal --- accumulator += (double)analogRead(SENSOR_PIN); sample_count++;
```

```
    // --- Make Decision --- if (sample_count >= decision_point) { double average_light_level = accumulator / decision_point; // --- State  
Machine Logic --- switch (currentState) { case WAITING_FOR_START: Serial.print("W."); // Waiting if (average_light_level >  
START_PULSE_THRESHOLD) { Serial.println("\n--- START PULSE DETECTED! ---"); Serial.println("Skipping rest of pulse & gap...");  
windows_to_skip = 3; currentState = SKIPPING_GAP; } break;
```

```
case SKIPPING_GAP: Serial.print("S-"); // Skipping windows_to_skip--; if (windows_to_skip <= 0) {  
Serial.println("\n--- Ready to Read Bits ---"); currentState = READING_BITS; bit_counter = 0; receivedByte = 0; // Clear the byte for  
the new char } break; case READING_BITS: // *** SIMPLE ON-OFF DECISION *** if (average_light_level >  
ON_OFF_THRESHOLD) { // It's a '1' bit Serial.print("1"); bitSet(receivedByte, 7 - bit_counter); // Build the byte MSB first } else {  
// It's a '0' bit Serial.print("0"); // No need to do anything, bit is already 0 bit_counter++; if (bit_counter >=  
MESSAGE_LENGTH) { // --- We have received all 8 bits --- char receivedChar = (char)receivedByte; // Convert the byte to a char  
Serial.print("> Char: "); Serial.println(receivedChar);
```

```
    // --- Packet Logic --- if (receivedChar == START_MARKER && !receivingMessage) { // This is the START of a new message  
receivingMessage = true; messageBuffer = ""; // Clear the buffer Serial.println("--- Start Marker Received ---"); } else if  
(receivedChar == END_MARKER && receivingMessage) { // This is the END of the message receivingMessage = false;  
Serial.println("\n--- End Marker Received ---"); Serial.println("FINAL MESSAGE:"); Serial.println(messageBuffer); // Print the complete,  
buffered message! Serial.println("-----\n"); } else if (receivingMessage) { // This is a normal character, add it  
to the buffer messageBuffer += receivedChar; } currentState = WAITING_FOR_START; // Go back to waiting for the next  
start pulse } break; }
```

```
    // Reset accumulators for the next window accumulator = 0.0; sample_count = 0; }
```

Software Implementation

Transmitter: TRANSMITTER V_OOK_PACKET

- `setup()`: Initializes LED_PIN as OUTPUT and starts it LOW.
- `loop()`:
 - Calls `send_char(startMarker)`.
 - Loops through the message string, calling `send_char()` for each letter.
 - Calls `send_char(endMarker)`.
 - Turns LED LOW and `delay(5000)` for a pause.
 - `send_char(char c)`:
 - `digitalWrite(HIGH) & delay(1500)`: Sends the 1.5s Start Pulse.
 - `digitalWrite(LOW) & delay(500)`: Sends the 0.5s Separation Gap.
 - Loops 8 times (for 8 bits, `i=7` down to 0).
 - `bitRead(c, i)` checks if the bit is 1 or 0.
 - `digitalWrite(HIGH/LOW) & delay(500)`: Sends the bit using OOK.

Software Implementation

Receiver: RECEIVER_VOOK_PACKET

- `setup()`: Initializes Serial and the `lastSampleTime` timer.
- `Loop()`:
 - Checks `micros()` to ensure it only runs every `timeBetweenSamples` (500μs), achieving a 2000 Hz sampling rate.
 - Adds `analogRead(SENSOR_PIN)` to an accumulator.
 - After `decision_point` (1000) samples:
 - `average_light_level = accumulator / decision_point`; (This is the Low-Pass FIR Filter / Convolution).
 - Enters a State Machine (`switch(currentState)`).

Software Implementation

State Machine:

- **WAITING_FOR_START:** If `avg_level > START_PULSE_THRESHOLD`, it sets `windows_to_skip = 3` and moves to **SKIPPING_GAP**.
- **SKIPPING_GAP:** Counts down `windows_to_skip`. When 0, it moves to **READING_BITS**.
- **READING_BITS:**
 - Loops 8 times (for 8 bits).
 - if (`avg_level > ON_OFF_THRESHOLD`): Decides '1' and uses `bitSet()` to build the `receivedByte`.
 - else: Decides '0'.
 - After 8 bits, it converts the byte to a char.
 - Packet Logic: Checks if the char is @ (start) or @ (end), or if it's a character to add to the `messageBuffer`.
 - Prints the FINAL MESSAGE if the end marker is received.
 - Resets accumulator and `sample_count` for the next 500ms window.

Results

The system successfully transmitted and decoded the full 25-character string. The receiver's Serial Monitor output clearly shows the process.

Tuning: The system was tuned by running debug code and observing the Avg Level for "ON" and "OFF" states, proving the hardware was stable. An ON_OFF_THRESHOLD of 200 was found to be optimal.

```
W...W...W...W...W...W...W...W...W...W
...
--- START PULSE DETECTED! ---
Skipping rest of pulse & gap...
S-S-S-
--- Ready to Read Bits ---
01000000 -> Char: @
--- Start Marker Received ---
W...
--- START PULSE DETECTED! ---
Skipping rest of pulse & gap...
S-S-S-
--- Ready to Read Bits ---
01001111 -> Char: O
W...
--- START PULSE DETECTED! ---
Skipping rest of pulse & gap...
S-S-S-
--- Ready to Read Bits ---
01110000 -> Char: p
W...
... (repeats for all characters) ...
W...
--- START PULSE DETECTED! ---
Skipping rest of pulse & gap...
S-S-S-
--- Ready to Read Bits ---
01101110 -> Char: n
W...
--- START PULSE DETECTED! ---
Skipping rest of pulse & gap...
S-S-S-
--- Ready to Read Bits ---
01000000 -> Char: @
--- End Marker Received ---
FINAL MESSAGE:
Optical Data Transmission
-----
```

Conclusion

Project Success

- A working optical data modem was successfully designed, built, and tested. The system reliably transmits a text string using IR light, a solar panel sensor, and a custom synchronization protocol.

Implementation of DSP Labs

- The project successfully implemented concepts from all six labs:
- DFT (FFT): Used critically in Phase 2 to analyze channel noise and disprove the FSK method due to hardware-induced harmonic cross-talk.
- FIR Filter (Lab 4): The final decoder's Moving Average Filter is a low-pass FIR filter, which was the correct DSP tool for the OOK signal.
- Convolution (Lab 2): The accumulator in the receiver code performs a linear convolution of the signal with the moving average filter's coefficients.
- IIR Filter (Lab 5) & Difference Equation (Lab 3): The dynamic DC offset removal used in our FSK debug code ($dc_average = 0.99 * \dots$) was a direct implementation of an IIR filter's difference equation.
- Decimation (Lab 6): The receiver implements decimation by a factor of 1000 (taking 1000 high-speed samples at 2000 Hz to produce one single output bit).

Conclusion

Key Findings

- The primary challenge was not software complexity but hardware limitations. The FSK (frequency) method failed because the solar panel's slow response and the transmitter's square-wave harmonics created unfixable signal interference. By systematically analyzing this with DSP tools (the FFT), we proved that a simpler modulation (OOK) combined with a stronger signal (IR LEDs) and a robust sync protocol was the superior engineering solution.

References

- ESP32 & ESP8266 Datasheets
- Arduino C++ Language Reference
- Random Nerd Tutorials: ESP32 ADC & Pinout Guides
- Digital Signal Processing: Principles, Algorithms, and Applications by Proakis & Manolakis
- T-Filter (<https://www.google.com/search?q=t-filter.appspot.com>) for FIR coefficient generation.
- Online resources for Optical Wireless Communication (OWC) and Li-Fi.